

ArchiCode



Alan Bissio

E.E.S.T. N°1 Vicente López

7mo 2da

Proyecto de Implementación de Sitios Web Dinámicos

Jorge Guzman

20 de Mayo de 2026

Índice

Introducción	2
Objetivos del Proyecto	3
Objetivo general	3
Objetivos específicos	3
Tecnologías Utilizadas	3
Frontend	3
Validación	3
Persistencia	4
Herramientas de desarrollo	4
Arquitectura de la Solución	4
Arquitectura hexagonal	4
Organización del proyecto	4
Persistencia local	5
Funcionalidades Implementadas	5
Registro de componentes	5
Validación de dependencias	5
Persistencia de datos	5
Eliminación de componentes	5
Decisiones Técnicas Importantes	5
Uso de TypeScript	5
Uso de Zod	6
Arquitectura hexagonal	6
Validación de patrones arquitectónicos	6
Uso de localStorage	6
Conclusión	7
Bibliografía	8

Introducción

ArchiCode es una aplicación web desarrollada en TypeScript orientada a la creación y validación de arquitecturas de software mediante componentes y dependencias.

El sistema permite registrar componentes arquitectónicos, establecer relaciones entre ellos y detectar errores de diseño utilizando reglas de validación arquitectónica.

El proyecto fue desarrollado utilizando una arquitectura hexagonal con separación por capas, permitiendo desacoplar la lógica principal de la interfaz y de los mecanismos de persistencia.

Para el desarrollo del frontend se utilizó Vite junto con TypeScript, mientras que las validaciones de datos fueron implementadas utilizando la librería Zod. La persistencia de información se realizó utilizando localStorage.

Además, el sistema implementa validaciones relacionadas con dependencias inexistentes, dependencias circulares y reglas arquitectónicas entre distintos tipos de componentes.

Objetivos del Proyecto

Objetivo general

Desarrollar una aplicación web dinámica que permita modelar arquitecturas de software mediante componentes y dependencias, aplicando conceptos de TypeScript, validaciones tipadas y arquitectura hexagonal.

Objetivos específicos

- Implementar una aplicación utilizando TypeScript estricto.
- Aplicar arquitectura hexagonal separando lógica de negocio, adaptadores e interfaz.
- Utilizar Zod para validar datos ingresados por el usuario.
- Implementar persistencia local mediante localStorage.
- Validar dependencias y relaciones entre componentes arquitectónicos.
- Detectar dependencias circulares y patrones arquitectónicos inválidos.
- Desarrollar una interfaz responsive y dinámica utilizando Vite.

Tecnologías Utilizadas

Frontend

- TypeScript
- Vite
- HTML
- CSS

Validación

- Zod

Persistencia

- localStorage

Herramientas de desarrollo

- GitHub
- Vercel
- Visual Studio Code

Arquitectura de la Solución

El proyecto fue desarrollado utilizando arquitectura hexagonal, separando la lógica de negocio de la interfaz y de los mecanismos de persistencia.

La estructura principal del proyecto fue organizada en distintas capas para facilitar el mantenimiento y la escalabilidad de la aplicación.

Arquitectura hexagonal

La lógica principal del sistema fue ubicada dentro de la carpeta “core”, donde se implementaron las entidades, puertos y casos de uso.

Los adaptadores y mecanismos de persistencia fueron implementados dentro de la carpeta “infraestructura”, permitiendo desacoplar el acceso a datos y la interfaz gráfica de la lógica principal.

Organización del proyecto

La aplicación fue organizada utilizando carpetas separadas para entidades, puertos, adaptadores, validaciones e interfaz de usuario.

Esta separación permitió mantener una estructura más ordenada y reutilizable.

Persistencia local

La persistencia de los componentes arquitectónicos fue implementada utilizando localStorage, permitiendo conservar la información incluso al recargar la página.

Funcionalidades Implementadas

Registro de componentes

- Creación de componentes arquitectónicos.
- Selección de tipo de componente.
- Registro de descripciones y dependencias.

Validación de dependencias

- Validación de dependencias inexistentes.
- Validación de dependencias circulares.
- Validación de relaciones inválidas entre componentes.

Persistencia de datos

- Almacenamiento local utilizando localStorage.
- Persistencia de componentes al recargar la página.

Eliminación de componentes

- Eliminación dinámica de componentes registrados.
- Actualización automática de la interfaz.

Decisiones Técnicas Importantes

Uso de TypeScript

Se decidió utilizar TypeScript debido a las ventajas relacionadas con el tipado estático, detección temprana de errores y mantenimiento del código.

Uso de Zod

Zod fue utilizado para implementar validaciones runtime sobre los datos ingresados por el usuario, permitiendo verificar nombres, descripciones y estructuras de componentes.

Arquitectura hexagonal

La aplicación fue desarrollada utilizando arquitectura hexagonal para desacoplar la lógica principal de la interfaz y de los mecanismos de persistencia.

Validación de patrones arquitectónicos

Se implementaron reglas de validación para detectar relaciones arquitectónicas inválidas entre componentes, como dependencias incorrectas entre frontend, APIs y bases de datos.

Uso de localStorage

localStorage fue utilizado para almacenar componentes arquitectónicos localmente sin necesidad de utilizar una base de datos externa.

Conclusión

El desarrollo de ArchiCode permitió aplicar conocimientos relacionados con TypeScript, validaciones tipadas, persistencia local y arquitectura hexagonal.

A lo largo del proyecto se trabajó con herramientas modernas de desarrollo frontend, implementando separación por capas y validaciones arquitectónicas sobre componentes y dependencias.

Además, el proyecto permitió comprender el uso de Zod para validaciones runtime, el uso de localStorage para persistencia de información y la importancia de mantener una estructura desacoplada utilizando puertos y adaptadores.

Como mejoras futuras podrían implementarse gráficos de dependencias, autenticación de usuarios, persistencia en bases de datos y visualización avanzada de arquitecturas.

Bibliografía

- Documentación oficial de TypeScript
<https://www.typescriptlang.org/>
- Documentación oficial de Vite
<https://vitejs.dev/>
- Documentación oficial de Zod
<https://zod.dev/>
- Documentación oficial de Vercel
<https://vercel.com/>
- GitHub
<https://github.com/>
- Documentación oficial de localStorage
<https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>
- Visual Studio Code
<https://code.visualstudio.com/>